

# MyMquid

BUG REPORT + FIX

## Dashboard Stats — Missing Fields

The admin dashboard's four stat cards (Total Posts, Published, Drafts, Scheduled) show blank values for Published, Drafts and Scheduled because `GET /dashboard/stats` only returns `totalPosts` and omits the other three fields.

Frontend — Fixed

Backend — Action Required

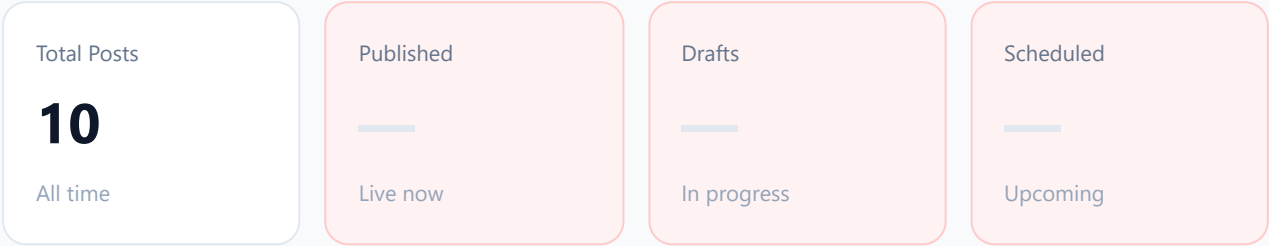
June 2026



# The Problem

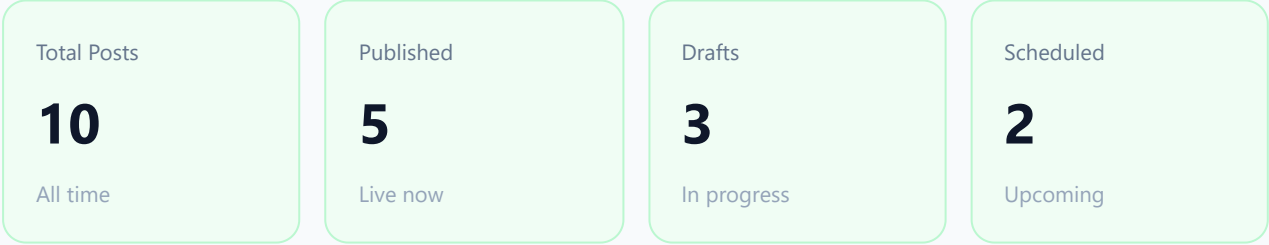
What the frontend sees vs. what it expects

## Before Fix — Three cards show blank values BUG



**Root cause:** When `stats.published`, `stats.drafts`, or `stats.scheduled` are `undefined`, React renders nothing — not even `0`. This happens because the backend omits those fields from the response.

## After Fix — All four cards display correctly FIXED



## What the Backend Currently Returns

### ACTUAL API RESPONSE (BROKEN)

```
{
  "totalPosts": 10
  // "published" is MISSING
  // "drafts" is MISSING
  // "scheduled" is MISSING
}
```

### EXPECTED API RESPONSE (CORRECT)

```
{
  "totalPosts": 10,
  "published": 5,
  "drafts": 3,
  "scheduled": 2
}
```

## Frontend Fix Applied (Temporary Fallback) DONE

The frontend now derives missing stat counts from the blog posts list as a fallback. When the backend starts returning the correct fields, the API values take priority automatically — no further frontend changes needed.

### SRC/ADMIN/DASHBOARD/DASHBOARDPAGE.TSX

```
// Before (broken – undefined values render as blank)
if (statsRes.status === "fulfilled") setStats(statsRes.value);

// After (fixed – derives missing fields from blog posts)
if (statsRes.status === "fulfilled") {
  const s = statsRes.value;
  setStats({
    totalPosts: s.totalPosts ?? s.total ?? posts.length,
    published: s.published ?? posts.filter(p => p.status === "published").length,
    drafts: s.drafts ?? posts.filter(p => p.status === "draft").length,
    scheduled: s.scheduled ?? posts.filter(p => p.status === "scheduled").length,
  });
}
```

**This fallback is a stopgap only.** The frontend fetches all blog posts to derive counts, which is inefficient at scale. The backend fix below removes this dependency entirely.



## Backend Fix Required

**ACTION NEEDED**

Add the three missing fields to GET /dashboard/stats

### What to Change

#### ENDPOINT

```
GET /api/v1/dashboard/stats
Authorization: Bearer <token>
```

#### REQUIRED RESPONSE SHAPE

```
{
  "totalPosts": 10,    // COUNT(*) from blog_posts
  "published": 5,      // COUNT(*) WHERE status = 'published'
  "drafts": 3,         // COUNT(*) WHERE status = 'draft'
  "scheduled": 2       // COUNT(*) WHERE status = 'scheduled'
}
```

### SQL Query

#### SINGLE QUERY APPROACH (MOST EFFICIENT)

```
SELECT
  COUNT(*) AS "totalPosts",
  COUNT(*) FILTER (WHERE status = 'published') AS "published",
  COUNT(*) FILTER (WHERE status = 'draft') AS "drafts",
  COUNT(*) FILTER (WHERE status = 'scheduled') AS "scheduled"
FROM blog_posts;
```

#### NESTJS SERVICE IMPLEMENTATION

```
async getStats(): Promise<DashboardStats> {
  const result = await this.blogPostsRepository
    .createQueryBuilder('post')
    .select('COUNT(*)', 'totalPosts')
    .addSelect("COUNT(*) FILTER (WHERE status = 'published')", 'published')
    .addSelect("COUNT(*) FILTER (WHERE status = 'draft')", 'drafts')
    .addSelect("COUNT(*) FILTER (WHERE status = 'scheduled')", 'scheduled')
    .getRawOne();

  return {
    totalPosts: parseInt(result.totalPosts, 10),
```

```
published: parseInt(result.published, 10),
drafts:    parseInt(result.drafts,    10),
scheduled: parseInt(result.scheduled, 10),
};
}
```

## TypeORM Alternative (if FILTER is not supported)

```
async getStats(): Promise<DashboardStats> {
  const [totalPosts, published, drafts, scheduled] = await Promise.all([
    this.blogPostsRepository.count(),
    this.blogPostsRepository.count({ where: { status: 'published' } }),
    this.blogPostsRepository.count({ where: { status: 'draft' } }),
    this.blogPostsRepository.count({ where: { status: 'scheduled' } }),
  ]);

  return { totalPosts, published, drafts, scheduled };
}
```

The `Promise.all` approach runs all 4 queries in parallel — negligible performance difference vs. the single query for typical post counts.

## Summary

| LAYER                             | STATUS     | ACTION                                                                                                                                  |
|-----------------------------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Frontend —<br>DashboardPage.tsx   | ✓ Fixed    | Derives missing counts from blog posts as fallback. No further changes needed.                                                          |
| Backend — GET<br>/dashboard/stats | ⚠ Required | Add <code>published</code> , <code>drafts</code> , <code>scheduled</code> fields to the response using COUNT queries grouped by status. |

Once the backend returns all four fields, the frontend fallback becomes inactive automatically — the API values take precedence via the `??` null-coalescing operator. No deployment of frontend changes will be needed.